



ORACLE

Base de données Manipulation des données

**Création et exploitation
de bases de données
420-315-AL**

Auteur :

Karim Mihoubi
Département d'informatique
Cégep André-Laurendeau
1111 Lapierre, Montréal (arr. LaSalle),
H8N 2J4
Tél: (514) 364-3320 [6162]
Karim.mihoubi@claurendeau.qc.ca



À voir...

ORACLE

- SELECT simple - Règles de traitement
- Algèbre relationnelle et base de données relationnelle
- Techniques d'indexage
- Avantages de la technique d'indexation
- Conséquences de l'indexation

Référence : SQL pour Oracle, Christian Soutou, EYROLLES
Pages 48-51 ; 616-631



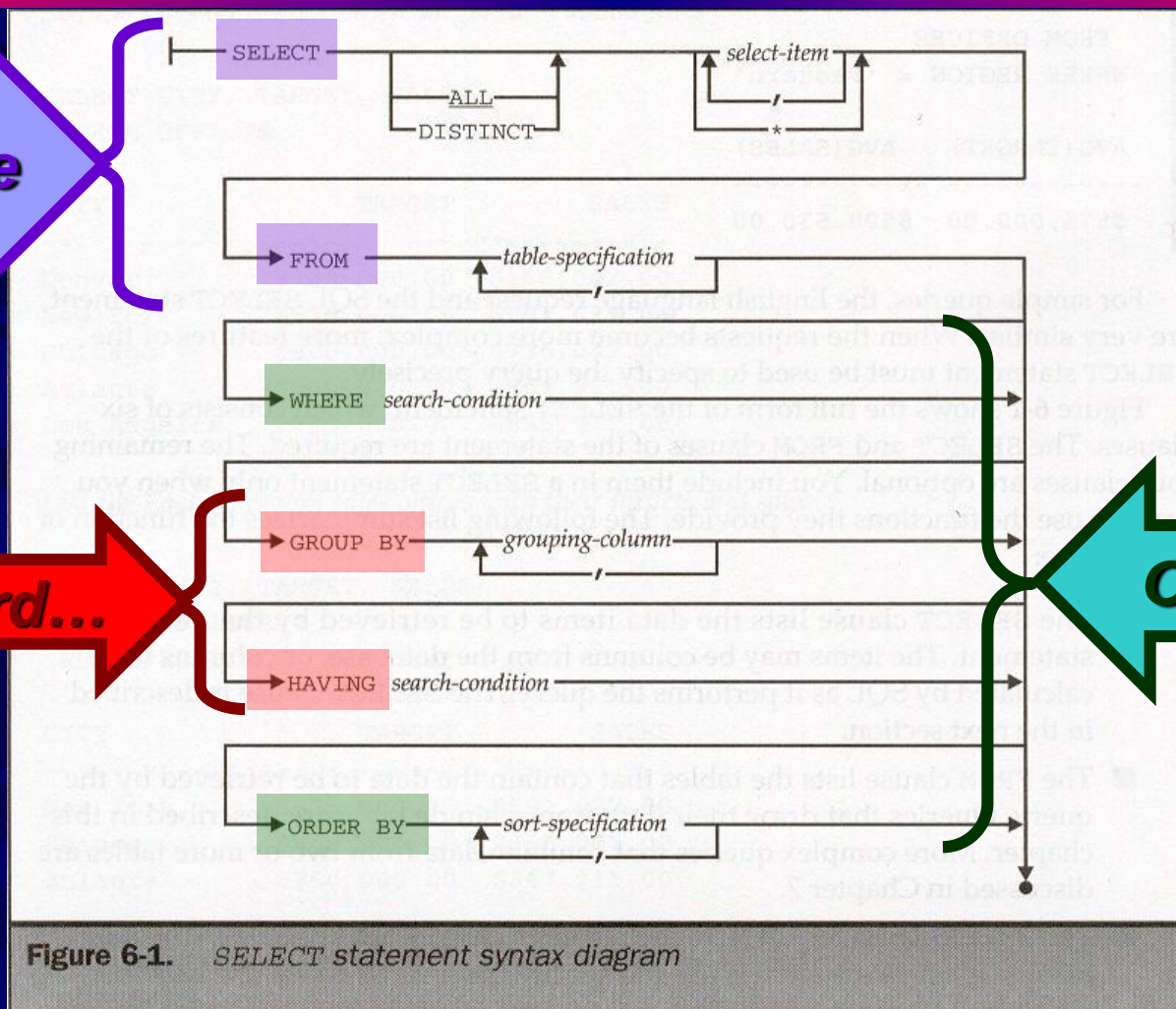
L'énoncé *SELECT*

ORACLE

Obligatoire

Plus tard...

Optionnel





L'énoncé *SELECT*

ORACLE

- *SELECT* liste des attributs à sélectionner
 - *FROM* nom de la table utilisée
 - *WHERE* critère(s) de sélection
 - *GROUP BY* colonne(s) de regroupements
 - *HAVING* critère(s) de sélection des groupes
 - *ORDER BY* ordre d'affichage des résultats



SELECT SUPPRIMER LES DOUBLONS

ORACLE

- Sauf indication contraire de votre part, la sortie d'une requête **SQL** affichera les résultats sans éliminer les enregistrements dont les valeurs ont déjà été affichées.
- Le SQL permet, grâce à la clause **DISTINCT**, de ne pas afficher les **doublons**
- Le mot réservé **DISTINCT** affecte toutes les colonnes de la liste et renvoie chaque combinaison distincte des colonnes de la clause **SELECT**
- Au risque d'avoir une erreur de compilation, Le mot clé **DISTINCT** doit apparaître directement après le mot clé **SELECT**



SELECT

TRIER LES DONNÉES

ORACLE

- Nous Les informations triées par ordre croissant sont familières à la plupart d'entre nous.
- Le **tri** facilite la recherche d'un numéro dans un annuaire téléphonique, la recherche d'un mot dans le dictionnaire ou la localisation d'une maison en fonction de son adresse.
- Le **SQL** utilise la clause **ORDER BY** pour ordonner les données.
- La clause **ORDER BY** peut être utilisée de plusieurs manières pour trier le résultat d'un **SELECT**
- La clause **ORDER BY** doit toujours être la dernière de la commande **SELECT**



SELECT

TRIER LES DONNÉES

ORACLE

- L'ordre de **tri** choisi par défaut est croissant.
 - Les **valeurs numériques** sont affichées du plus bas au plus élevé.
 - Les valeurs de type **date** sont affichées de la plus ancienne à la plus récente.
 - Les valeurs de type **caractères** sont affichées par ordre alphabétique.
 - Les valeurs **NULL** sont affichées en dernier si l'ordre d'affichage est croissant (par défaut), et en premier si l'ordre d'affichage choisi est décroissant.
 - La clause **NULLS FIRST** demande à Oracle que les valeurs NULL soient retournées avant les valeurs non NULL.
 - La clause **NULLS LAST** demande à Oracle que les valeurs doivent être renvoyées après les valeurs non NULL.



SELECT TRIER LES DONNÉES

ORACLE

```
SELECT last_name, hire_date  
FROM employees  
ORDER BY hire_date;
```

```
SELECT last_name, hire_date  
FROM employees  
ORDER BY hire_date DESC;
```

```
SELECT last_name, hire_date AS "Date de recrutement"  
FROM employees  
ORDER BY "Date de recrutement";
```

- Il est également permis de trier en utilisant une colonne ou expression qui n'est pas présente parmi les éléments sélectionnés

```
SELECT employee_id, first_name  
FROM employees  
WHERE employee_id < 105  
ORDER BY last_name;
```




SELECT TRIER LES DONNÉES

ORACLE

- Il est également possible de trier le résultat de l'exécution de la requête sur plus qu'une colonne.

```
SELECT department_id, last_name  
FROM employees  
WHERE department_id <= 50  
ORDER BY department_id, last_name;
```

```
SELECT department_id, last_name  
FROM employees  
WHERE department_id <= 50  
ORDER BY department_id DESC, last_name;
```



Règles de traitement d'une requête simple

ORACLE

1. Sélection de la table dans la clause **FROM**
2. S'il y a une clause **WHERE**, extraire tous les uplets pour lesquels la condition est **VRAI**. *Les uplets, où la condition est FAUSSE ou NULL, sont rejetés*
3. Pour les uplets restants, extraire les attributs spécifiés (*champs de référence ou champs calculés*) par l'énoncé **SELECT**



Règles de traitement d'une requête simple

ORACLE

4. Si la clause **DISTINCT** est présente, éliminer les uplets dupliqués
5. Si la clause **ORDER BY** est présente, trier les uplets selon les spécifications

Les uplets générés par cette procédure représentent le résultat de la requête



Opérations ensemblistes

- **L'algèbre relationnelle** utilise cinq catégories de commandes pour opérer sur les relations.
 - La projection et la restriction
 - Les opérations ensemblistes (union, différence, intersection)
 - Le produit cartésien
 - Les jointures
 - L'agrégation



Le SQL

ORACLE

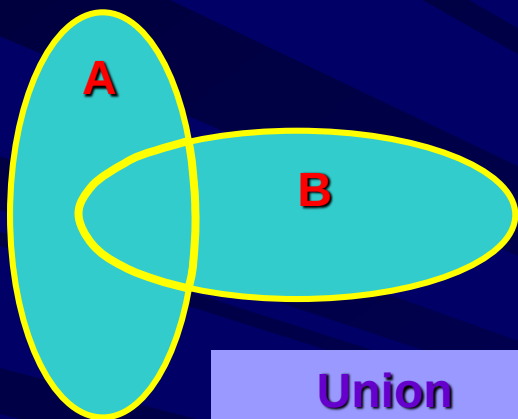
Opérations ensemblistes

- L'**algèbre relationnelle** et ses **opérateurs ensemblistes** permettent de combiner les résultats de plusieurs requêtes distinctes pour former une seule **relation** (table).
- Les cinq opérations fondamentales sont :
 - Union
 - Différence
 - Jointure
 - Intersection
 - Division

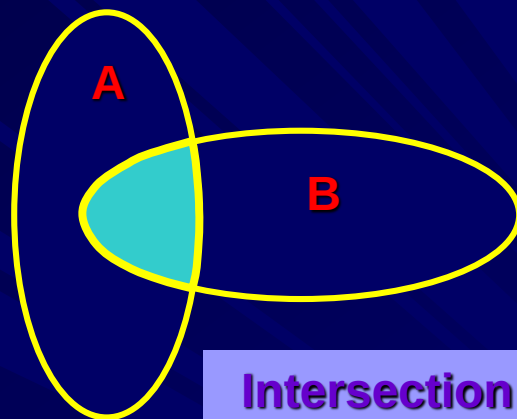


Opérations ensemblistes

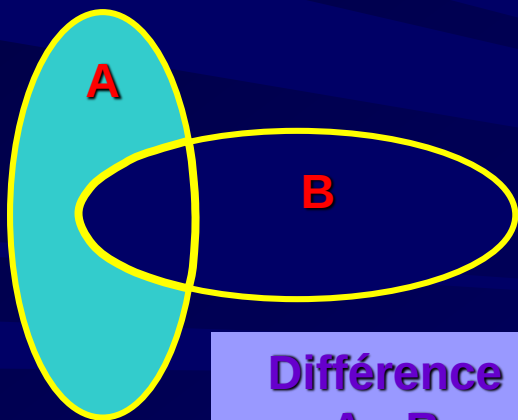
ORACLE



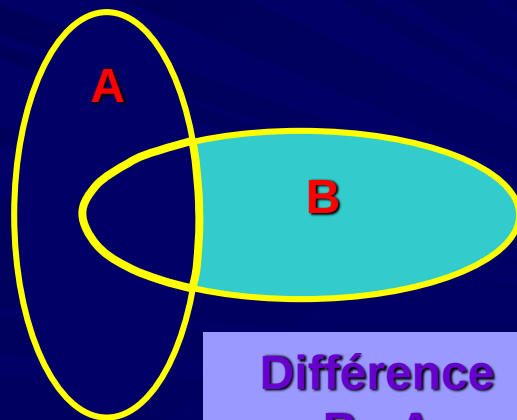
Union
 $A \cup B$



Intersection
 $A \cap B$



Différence
 $A - B$



Différence
 $B - A$



Opérations ensemblistes (*UNION*)

ORACLE

- L'opérateur **UNION** permet de *fusionner* les résultats de deux requêtes distinctes
- Les opérateurs UNION et UNION ALL sont commutatifs

SELECT1...

UNION [ALL | DISTINCT]

[CORRESPONDING [BY](liste colonnes)]

SELECT2...

[Algèbre relationnel – lien utile](#)



Opérations ensemblistes (*UNION*)

ORACLE

- Par défaut, l'union enlève les doublons (comme le font **DISTINCT** ou **UNIQUE** dans un **SELECT**), excepté pour les valeurs **NULL** (clause **DISTINCT**)
- Pour récupérer tous les uplets, on utilise la clause **ALL**
- La clause **CORRESPONDING** permet de limiter la portée de l'union aux colonnes de même nom (Oracle)

[Algèbre relationnel – lien utile](#)



Opérations ensemblistes (UNION)

ORACLE

On veut afficher tous les produits pour lesquels
il y a une commande au-dessus de \$30 000

OU

il y a plus de \$30 000 en inventaire

```
SELECT COM_USINE, COM_PRODUIT
FROM EX1_COMMANDE_COM
WHERE COM_MONTANT > 30000
```

COM_USINE	COM_PRODUIT
REI	2A44L
REI	2A44R
IMM	775C

```
SELECT PRO_NOUSINE, PRO_NOPRODUIT
FROM EX1_PRODUIT_PRO
WHERE (PRO_PRIX * PRO_QTESTOCK) > 30000;
```

PRO_NOUSINE	PRO_NOPRODUIT
ACI	4100Y
REI	2A44L
ACI	4100Z
REI	2A44R

```
SELECT COM_USINE, COM_PRODUIT
FROM EX1_COMMANDE_COM
WHERE COM_MONTANT > 30000
UNION
SELECT PRO_NOUSINE, PRO_NOPRODUIT
FROM EX1_PRODUIT_PRO
WHERE (PRO_PRIX * PRO_QTESTOCK) > 30000;
```

COM_USINE	COM_PRODUIT
ACI	4100Y
ACI	4100Z
IMM	775C
REI	2A44L
REI	2A44R



Opérations ensemblistes (UNION)

ORACLE

- Pour faire apparaître tous les uplets de l'opération, il faut utiliser la clause **UNION ALL**

On veut afficher tous les produits pour lesquels
il y a une commande au-dessus de \$30 000

OU

il y a plus de \$30 000 en inventaire

```
SELECT COM_USINE, COM_PRODUIT
FROM EX1_COMMANDE_COM
WHERE COM_MONTANT > 30000
UNION ALL
SELECT PRO_NOUSINE, PRO_NOPRODUIT
FROM EX1_PRODUIT_PRO
WHERE (PRO_PRIX * PRO_QTESTOCK) > 30000;
```

[Algèbre relationnel – lien utile](#)

COM_USINE	COM_PRODUIT
REI	2A44L
REI	2A44R
IMM	775C
ACI	4100Y
REI	2A44L
ACI	4100Z
REI	2A44R



Opérations ensemblistes (*UNION*)

ORACLE

- Il est possible de **trier** les résultats de la requête
- La clause **ORDER BY** doit apparaître une seule fois et à la fin de la requête
- L'ordre de tri s'applique sur les colonnes de la première requête
- L'ordre de tri croissant choisi par défaut, sauf avec l'opérateur **UNION ALL**.



Opérations ensemblistes (*UNION*)

ORACLE

On veut afficher tous les produits pour lesquels
il y a une commande au-dessus de \$30 000
OU
il y a plus de \$30 000 en inventaire

```
SELECT COM_USINE, COM_PRODUIT
FROM EX1_COMMANDE_COM
WHERE COM_MONTANT > 30000
UNION
SELECT PRO_NOUSINE, PRO_NOPRODUIT
FROM EX1_PRODUIT_PRO
WHERE (PRO_PRIX * PRO_QTESTOCK) > 30000
ORDER BY 1,2 DESC;
```

COM_USINE	COM_PRODUIT
ACI	4100Z
ACI	4100Y
IMM	775C
REI	2A44R
REI	2A44L



Opérations ensemblistes (*INTERSECTION*)

ORACLE

- L'opérateur **INTERSECT** permet de récupérer les *résultats communs* de deux requêtes distinctes

SELECT1...

INTERSECT [ALL | DISTINCT]

[CORRESPONDING [BY](liste colonnes)]

SELECT2...



Opérations ensemblistes (*INTERSECTION*)

ORACLE

- Par défaut, l'intersection enlève les doublons, excepté pour les valeurs **NULL** (clause **DISTINCT**)
- Pour récupérer tous les uplets, on utilise la clause **ALL**
- La clause **CORRESPONDING** permet de limiter la portée de l'intersection aux colonnes de même nom



Opérations ensemblistes (INTERSECTION)

ORACLE

On veut afficher tous les produits pour lesquels
il y a une commande au-dessus de \$30 000
ET
il y a plus de \$30 000 en inventaire

```
SELECT COM_USINE, COM_PRODUIT
FROM EX1_COMMANDE_COM
WHERE COM_MONTANT > 30000
```

COM_USINE	COM_PRODUIT
REI	2A44L
REI	2A44R
IMM	775C

```
SELECT PRO_NOUSINE, PRO_NOPRODUIT
FROM EX1_PRODUIT_PRO
WHERE (PRO_PRIX * PRO_QTESTOCK) > 30000;
```

PRO_NOUSINE	PRO_NOPRODUIT
ACI	4100Y
REI	2A44L
ACI	4100Z
REI	2A44R

```
SELECT COM_USINE, COM_PRODUIT
FROM EX1_COMMANDE_COM
WHERE COM_MONTANT > 30000
INTERSECT
SELECT PRO_NOUSINE, PRO_NOPRODUIT
FROM EX1_PRODUIT_PRO
WHERE (PRO_PRIX * PRO_QTESTOCK) > 30000;
```

COM_USINE	COM_PRODUIT
REI	2A44L
REI	2A44R



Opérations ensemblistes (DIFFÉRENCE)

ORACLE

- L'opérateur **MINUS** (*En Oracle*) [**EXCEPT en SQL2**] permet de récupérer des *données présentes dans une table et sans correspondance dans l'autre table.*

```
SELECT1...  
MINUS [ALL | DISTINCT]  
      [CORRESPONDING [BY](liste colonnes)]  
SELECT2...
```



Opérations ensemblistes (*DIFFÉRENCE*)

ORACLE

- Le résultat sera différent dépendant de la table qui sera nommée en premier
- Par défaut, la différence enlève les doublons, excepté pour les valeurs **NULL** (clause **DISTINCT**)
- Pour récupérer tous les uplets, on utilise la clause **ALL**
- La clause **CORRESPONDING** permet de limiter la portée de la différence aux colonnes de même nom



Opérations ensemblistes (DIFFÉRENCE)

ORACLE

On veut afficher tous les produits pour lesquels
il y a une commande dont le montant se situe au-dessus de \$30 000
EXCEPTÉ
pour les produits dont le prix est supérieur à \$3 000

```
SELECT COM_USINE, COM_PRODUIT  
FROM EX1_COMMANDE_COM  
WHERE COM_MONTANT > 30000
```

COM_USINE	COM_PRODUIT
REI	2A44L
REI	2A44R
IMM	775C

```
SELECT PRO_NOUSINE, PRO_NOPRODUIT  
FROM EX1_PRODUIT_PRO  
WHERE PRO_PRIX > 3000;
```

PRO_NOUSINE	PRO_NOPRODUIT
REI	2A44L
REI	2A44R

```
SELECT COM_USINE, COM_PRODUIT  
FROM EX1_COMMANDE_COM  
WHERE COM_MONTANT > 30000  
MINUS  
SELECT PRO_NOUSINE, PRO_NOPRODUIT  
FROM EX1_PRODUIT_PRO  
WHERE PRO_PRIX > 3000;
```

COM_USINE	COM_PRODUIT
IMM	775C



Opérations ensemblistes

Règles

ORACLE

- Instruction_select1 et instruction_select2 représentent des instructions **SELECT** valides.
- Si une clause **ORDER BY** est utilisée, elle doit faire référence au numéro de la colonne plutôt qu'à son nom, car le nom peut être différent dans chacun des ordres SELECT
- Le **nombre de colonnes** renvoyées par instruction_select1 doit être égal au nombre de colonnes renvoyées par instruction_select2.



Opérations ensemblistes

Règles

ORACLE

- Seules des colonnes de même type (**CHAR**, **VARCHAR2**, **DATE** ou **NUMBER**) doivent être comparées avec des opérateurs ensemblistes
- Les **types de données** des colonnes de instruction_select1 doivent correspondre à ceux des colonnes de instruction_select2.
- Les noms pris en compte de colonnes (titres) sont ceux choisis lors du premier SELECT
- La largeur des colonnes prise en compte est celle qui est la plus grande parmi tous les ordres SELECT.
- Aucun attribut ne peut avoir le type LONG



EXERCICES #1

ORACLE





ORACLE

SQL et notions d'index

**Création et exploitation
de bases de données
420-315-AL**

Auteur :

Karim Mihoubi
Département d'informatique
Cégep André-Laurendeau
1111 Lapierre, Montréal (arr. LaSalle),
H8N 2J4
Tél: (514) 364-3320 [6162]
Karim.mihoubi@claurendeau.qc.ca



À voir...

ORACLE

- Présentation
- Types d'index
- Techniques d'indexage
- Avantages de la technique d'indexation
- Conséquences de l'indexation

Référence : SQL pour Oracle, Christian Soutou, EYROLLES
Pages 48-51 ; 616-631



Présentation



- **Index** : objet base de données permettant «d'indexer» une ou plusieurs colonnes dans le but d'améliorer ou de valider l'accès aux données par le SGBDR.
- Un **index** peut être créer implicitement ou explicitement par un utilisateur.
- Oracle crée un index automatiquement à chaque fois qu'une **clé primaire** ou qu'une **clé unique** (candidate) est créée.
- Un **index** ne peut pas être composé de plus de 16 colonnes avec Oracle



Présentation

ORACLE

- Un **index** ne stocke jamais de valeur **NULL** et donc si un attribut pouvant contenir une valeur **NULL** est indexé, la requête qui sollicite ce dernier dans une clause **WHERE** ne pourra pas exploiter cette technique.
- Le langage **SQL** ne propose pas d'instructions permettant d'examiner le contenu d'un index.
- Liens utiles :
 - [Optimisation de requête](#)
 - [Indexer NULL](#)



Avantages-Inconvénients

ORACLE

- ***Avantages de la technique d'indexation***
 - Augmentation de la vitesse d'exécution des requêtes.
 - S'assure qu'une colonne donnée ne dispose que d'une seule valeur (quand unique).
- ***Conséquences de l'indexation***
 - La création d'index implique l'utilisation d'espace mémoire dans la base de données, et, étant donné qu'il est mis à jour à chaque modification de la table à laquelle il est rattaché, peut alourdir le temps (écriture sur disque) de traitement du SGBDR lors de la saisie de données.



Avantages-Inconvénients

ORACLE

- **Pourquoi créer l'index ?** Sans index, toute recherche s'apparente à un parcours **séquentiel** de toute la table. Si nous avons n lignes, prévoir un nombre de lecture moyen de $n/2$ (n nombre de ligne)
- **Quand faut-il penser créer un index ?**
 - Inutile de créer des **indexes** quand la taille de la table est petite (table de moins de 300 enregistrements).
 - Indexer plutôt les colonnes que l'on retrouvent souvent dans les clauses **WHERE** et **ORDER BY**.
 - Indexer les colonnes impliquées dans les jointures (clé primaire et clé étrangère)



Types d'index

ORACLE

- On distingue deux types d'index :

a) **Index unique** : permet de retrouver les contraintes de clé primaire et de clé unique. Ils sont créés implicitement avec les **clés primaire** et **clés unique**.

```
CREATE INDEX { UNIQUE | BITMAP } [schéma.]nomIndex  
ON [schéma.]nomTable ( {colonne1 | expressionColonne1}  
[ASC | DESC ...]); }
```

Exemple :

```
CREATE UNIQUE INDEX id_cpteEpargne  
ON CompteEpargne (titulaire DESC) ;
```



Types d'index

ORACLE

- On distingue deux types d'index :
 - a) **Index non unique** : si l'option UNIQUE n'est pas présente, l'index permet les doublons. Index créé pour permettre l'amélioration des performances des requêtes. On le retrouvent associés aux clauses WHERE et HAVING les **clés primaire** et **clés unique**.

```
CREATE INDEX { UNIQUE | BITMAP } [schéma.]nomIndex  
ON [schéma.]nomTable ( {colonne1 | expressionColonne1}  
[ASC | DESC ...]); }
```

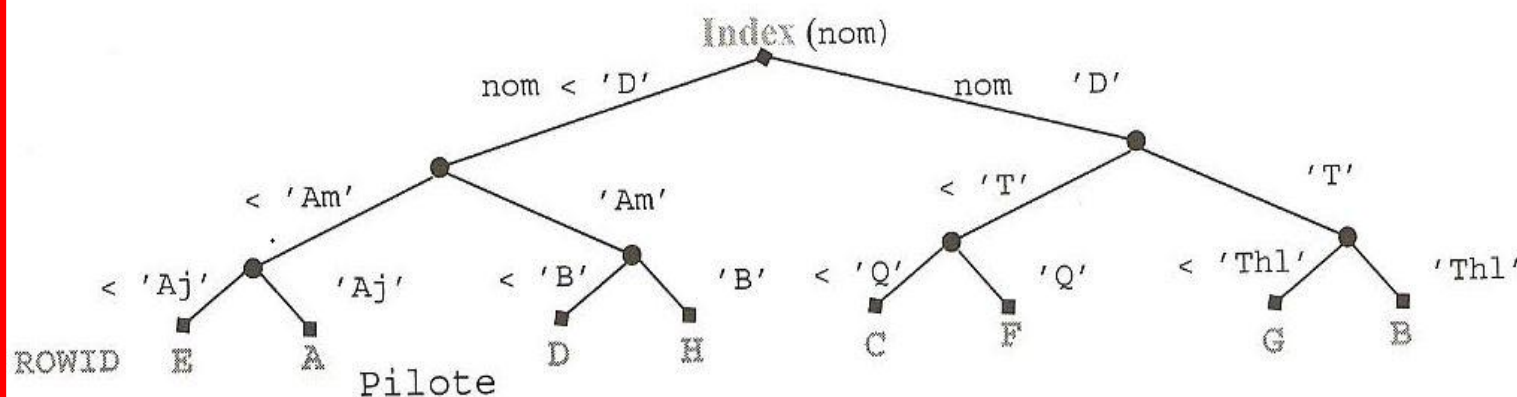
```
CREATE INDEX ind_NomClient  
ON Pilote (nom DESC) ;
```




Exemple

ORACLE

Figure 1-3 Index sur la colonne nom



ROWID	brevet	nom	nbHVol	compa
A	PL-1	Amélie Sulpice	450	AF
B	PL-2	Thomas Sulpice	900	AF
C	PL-3	Paul Soutou	1000	SING
D	PL-4	Aurélia Ente	850	ALIB
E	PL-5	Agnès Bidal	500	SING
F	PL-6	Sylvie Payrissat	2500	SING
G	PL-7	Thierry Guibert	600	ALIB
H	PL-8	Cathy Castaings	400	AF

SQL
Pour
Oracle
Christian
Soutou
Page 51



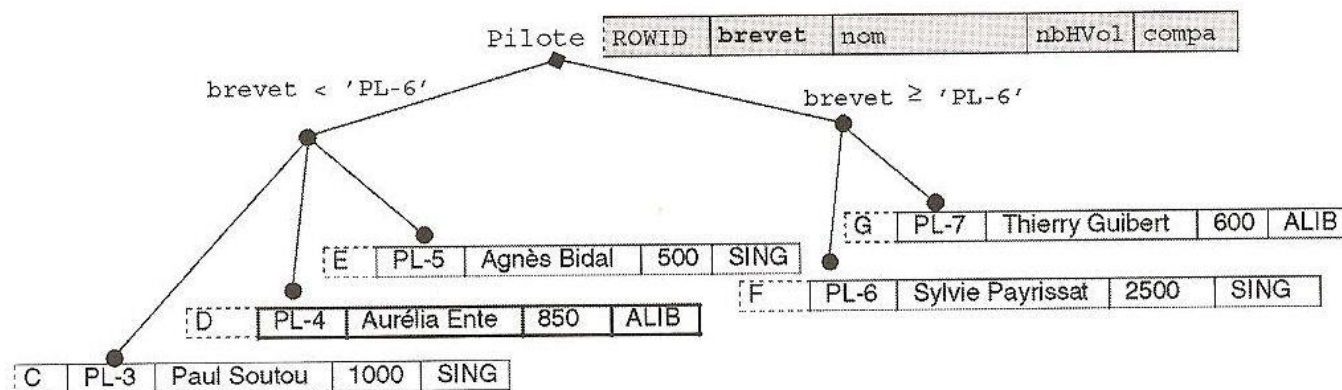
Exemple

ORACLE

Tables ordinaires	Tables organisées en index
La pseudo-colonne ROWID identifie chaque enregistrement. La clé primaire est optionnelle.	La clé primaire est obligatoire pour identifier chaque enregistrement.
Le ROWID physique permet de construire des index secondaires.	Le ROWID logique permet de construire des index secondaires.
Utilisation de clusters possible.	Utilisation interdite de clusters.
Peut contenir une colonne de type LONG et plusieurs colonnes de type LOB.	Peut contenir plusieurs colonnes LOB mais aucune de type LONG.

La figure suivante illustre la table `Pilote` organisée en index basé sur la clé primaire `brevet`.

Figure 1-6 Table organisée en index



SQL
Pour
Oracle
Christian
Soutou
Page 51



Types d'index

ORACLE

- **On distingue les indexes :**
 - a) **Index dense** : plus rapide d'accès, mais plus gourmand en mémoire et revient plus chère lors d'une suppression et d'une insertion d'enregistrement. Nombre de clés égal au nombre d'articles dans le fichier
 - b) **Index non dense** : nécessite moins d'espace mémoire, les opérations d'insertion et de suppression reviennent moins chère. Dans ce type d'index, on ne crée des enregistrements index que pour certains enregistrements (bloc d'enregistrements, index non dense par bloc). Pour localiser un enregistrement, on utilise un algorithme jusqu'à se rapprocher le plus de cet enregistrement, puis la recherche se fait séquentiellement.



Index dense

ORACLE

- **Index dense** : plus rapide d'accès, mais plus gourmand en mémoire et revient plus chère lors d'une suppression et d'une insertion d'enregistrement. Nombre de clés égal au nombre d'articles dans le fichier.

Exemple : Soit un fichier agence, à chaque agence correspond un certain nombre de clients. En utilisant ce type d'index, l'index dense nous envoie directement au premier enregistrement relatif à cette agence. Puis, grâce à son pointeur, on accède au second enregistrement.

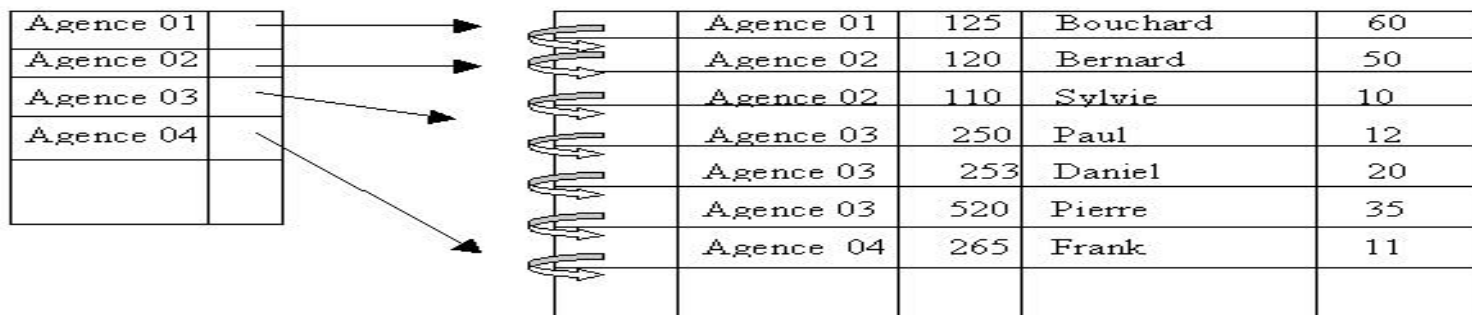


FIGURE : Index Dense



Index non dense

ORACLE

- **Index non dense** : nécessite moins d'espace mémoire, les opérations d'insertion et de suppression reviennent moins chère. Dans ce type d'index, on ne crée des enregistrements index que pour certains enregistrements (bloc d'enregistrements, index non dense par bloc). Pour localiser un enregistrement, on utilise un algorithme jusqu'à se rapprocher le plus de cet enregistrement, puis la recherche se fait séquentiellement.



Techniques d'indexages

ORACLE

- **Techniques d'indexage** : il en existe plusieurs. Chacune s'adapte bien à un type donné de problème à résoudre. On les évalue généralement au moyen des paramètres suivants :
 - **Temps d'accès** : temps nécessaire pour accéder à une donnée particulière.
 - **Délai d'insertion** : mise en place de la donnée dans la BD et mise à jour du fichier index.
 - **Délai d'effacement** : Temps nécessaire pour supprimer une donnée et mettre à jour le fichier index.
 - **Occupation mémoire** : volume mémoire nécessaire au stockage des index.



Techniques d'indexages

ORACLE

Les deux techniques les plus utilisées sont :

1. La technique ISAM (Indexed Sequential Access Method - IBM)
2. La technique par arbres : il en existent plusieurs, la plus célèbre (B+ qu'utilise Oracle).



Techniques d'indexages

B-tree(B+)

ORACLE

- Technique par arbre B+ (Oracle, VSAM, INGRES) : les techniques séquentiels indexés ont pour désavantage de perdre leurs performances lorsque leur taille grandit. La technique par arbre B+ permet de surmonter ce problème. Le principe est basé sur quelques règles qui sont :
 - Indexage à plusieurs niveaux.
 - Chaque nœud doit contenir au moins $n/2$ pointeurs à tous les niveaux, sauf pour sa racine (n étant une valeur fixée pour un arbre).
 - Chaque nœud de l'arbre possède de $n/2$ à n enfants.
 - Un nœud terminal comprends $(n - 1)$ valeurs de clés de tri.



-

The diagram illustrates a hierarchical tree structure for a 1D bin packing problem. The root node, labeled "Racine", contains the values 50, 100, and three empty slots. It branches into three child nodes. The left child node contains 16, 23, 50, and one empty slot. The middle child node contains 57, 85, 100, and one empty slot. The right child node contains 117, 145, and two empty slots. A horizontal dashed line separates the "Racine" level from the "Niveau terminal" level. Below this line, the left child node branches into four terminal nodes: [05, 11, 16], [18, 20, 23], [35, 40, 50], and [52, 55, 57]. The middle child node branches into four terminal nodes: [66, 70, 85], [90, 95, 100], [111, 113, 117], and [145]. The right child node has one terminal child: [90, 95, 100]. Arrows indicate the flow from parent nodes to their respective child nodes.



EXERCICES #2

ORACLE

